

Qasim Sethar

AI Systems Engineer · Autonomous Agents · Retrieval Infrastructure · Applied ML

qasimio.me | linkedin.com/in/qasimio | github.com/qasimio | amkassim444@gmail.com

Professional Summary

AI Systems Engineer with hands-on experience architecting autonomous agent loops, retrieval-augmented generation pipelines, and production developer tooling. Built **Operon** — a terminal-native autonomous coding agent with a persistent cross-file symbol graph, AST-based refactoring engine, and a deterministic verification layer that eliminates the most common failure modes of local LLMs. Published **foldr** on PyPI — a cross-platform file automation CLI with background daemon watch mode, full undo, and support for 240+ file extensions across 30+ categories. Strong foundation in classical information retrieval applied directly to practical RAG systems. Designs software where safety and correctness are first-class engineering constraints.

Technical Skills

Languages: Python (4 yrs), Java (2 yrs), C++ (1.5 yrs), SQL, Bash

Agentic AI: ReAct Architecture, Tool Calling, Multi-step Planning, Self-correction, Human-in-the-Loop Oversight

LLM & ML: LlamaIndex, LangChain, HuggingFace Transformers, PyTorch, BERT Fine-tuning, Cross-Encoders

Retrieval: RAG Pipelines, Hybrid Search (Sparse+Dense), LanceDB, ChromaDB, FastEmbed, Inverted Indices, Reranking

Parsing / AST: Python `ast`, `tokenize`, Tesseract OCR, Poppler, Document Intelligence

Systems: Linux, Docker, GitHub Actions, Multithreading, Daemon Processes, `inotify/kqueue/ReadDirectoryChangesW`, PyPI Packaging

Tooling: `argparse`, `watchdog`, `setuptools`, `wheel`, `entry_points`, Textual (TUI), Click, Git

Engineering Projects

Operon — Autonomous Code Intelligence Agent

Python · Textual · LLM APIs · AST · ReAct

<https://github.com/qasimio/Operon>

- Architected a terminal-native autonomous agent that builds a persistent, hash-gated symbol graph across an entire repository — enabling cross-file refactoring, documentation generation, and execution flow analysis without loading full files into context.
- Engineered a **deterministic-first REVIEWER** that verifies file changes by comparing disk content against diff memory snapshots before any LLM call, eliminating hallucinated change confirmation — the root cause of most agent loop failures.
- Implemented a **CRUD fast-path** using Python `ast` and `tokenize` for structured operations (import insertion, variable renaming, comment placement), removing LLM dependency for predictable edits and cutting noop failure rate to zero for these cases.
- Designed a **5-tier surgical diff engine** with cascading matching from exact string to whitespace-normalized to fuzzy multi-line, enabling reliable SEARCH/REPLACE patching against files modified by both LLM and deterministic paths.
- Built a universal **LLM router supporting 9 providers** (local, OpenAI, Anthropic, OpenRouter, Deepseek, Groq, Together, Azure, custom) with hot-reload config — model switching takes effect on the next call, no process restart required.
- Implemented mandatory **approval gate architecture**: no filesystem write occurs without explicit human confirmation; 300s timeout auto-rejects to prevent thread hang in the multithreaded Textual TUI.

MQNotebook — Enterprise RAG System

Python · LlamaIndex · Tesseract · Poppler · Streamlit

<https://github.com/qasimio/MQNotebook>

- Engineered a local-first RAG pipeline targeting enterprise document formats excluded by most RAG implementations: scanned PDFs (image-only pages), PPTX speaker notes, and multi-sheet XLSX — using a custom Tesseract + Poppler OCR ingestion layer.
- Implemented **hybrid retrieval** combining dense vector search with a Cross-Encoder reranker, achieving **~40% precision improvement** over naive cosine similarity in context selection accuracy.
- Solved persistent WinError 32 file-locking failures in ChromaDB on Windows by designing a **session-isolated storage handler** that dynamically routes each session to a separate persistent store, eliminating cross-session lock contention.
- Optimized context injection strategy, **reducing token consumption by ~60%** without degrading retrieval quality — enabling deployment within Streamlit Cloud resource constraints while maintaining full document coverage.

DevShelf — Vertical Search Engine (from First Principles)

Java · Information Retrieval · Data Structures

<https://github.com/qasimio/DevShelf>

- Architected a full vertical search engine for CS literature **without Lucene or ElasticSearch** — implementing a custom Positional Inverted Index with **O(1)** keyword retrieval via direct-address hashing.

- Engineered an Offline Indexer that decouples corpus processing from query time, achieving **sub-millisecond** online query latency. Implemented O(L) Trie for autocomplete; Levenshtein Distance for fuzzy correction.
- Classical IR architecture from DevShelf directly informed the hybrid search and reranking design in MQNotebook — demonstrating intentional, theory-grounded engineering progression.

FOLDER — File Automation CLI [pip install folder](#)
<https://github.com/qasimio/folder>

Python · PyPI · watchdog · Daemon Processes · Cross-platform

- Designed and published a production-ready cross-platform file automation CLI to PyPI — classifying **240+ file extensions** across **30+ categories** with custom TOML-based configuration, targeting data pipeline preparation and ML dataset cleaning workflows.
- Engineered a **background daemon architecture** using OS-native filesystem event APIs — inotify (Linux), kqueue (macOS), ReadDirectoryChangesW (Windows) via the `watchdog` library — achieving 0% CPU overhead when idle; daemon survives terminal close and runs indefinitely.
- Built a **two-phase watch pipeline**: initial full-directory organize pass on daemon start, followed by continuous event-driven organization — files moved back to root are re-organized; no stale session state blocks re-processing.
- Implemented a **JSON undo system**: every organize operation writes an immutable history entry; operations are undone independently without sequential rollback — git-revert semantics applied to filesystem state.
- Designed **dry-run architecture** that previews the complete planned I/O before any file is touched, with automatic conflict resolution (`file_(1).ext` naming) and explicit directory protection — zero implicit deletes.
- Mastered Python packaging standards (`setuptools`, `wheel`, `entry_points`) and CI/CD release pipeline to PyPI; zero external dependencies for terminal output — ANSI via `ctypes` (Win10+) with `colorama` fallback.

Experience

Machine Learning Intern · Arch Technologies (Remote)

Present

- Fine-tuned BERT models for NLP text classification tasks; profiled and optimized preprocessing pipelines, materially improving batch throughput for downstream training runs.
- Integrated trained PyTorch models into internal production prototypes in collaboration with senior engineering teams, including model serialization, versioning, and serving infrastructure.

Open Source & Published Tools

- **DevShelf** · Search Engine — Blazing-fast offline search for 250+ CS books — built from scratch in Java. [Download DevShelf](#)
- **Operon** · GitHub — Open-source autonomous code intelligence agent github.com/qasimio/Operon
- **FOLDER v2.1** · PyPI — Cross-platform file automation CLI; 240+ extensions, watch mode, undo system pypi.org/project/folder

Additional Technical Work

BabyGPT — Character-level LSTM language model from scratch (TensorFlow); demonstrates sequence modeling understanding prior to transformer usage. **MQ Banking Core** — Transactional banking system in C++ with file-level I/O and balance integrity guarantees. **Digital Eye** — CNN-based image classification pipeline (Keras/TensorFlow).

Education

B.Sc. Computer Science — Distributed Computing & AI Systems · Sukkur IBA University, Pakistan

Expected 2028